

LoRa hands-on #2

Number Guess



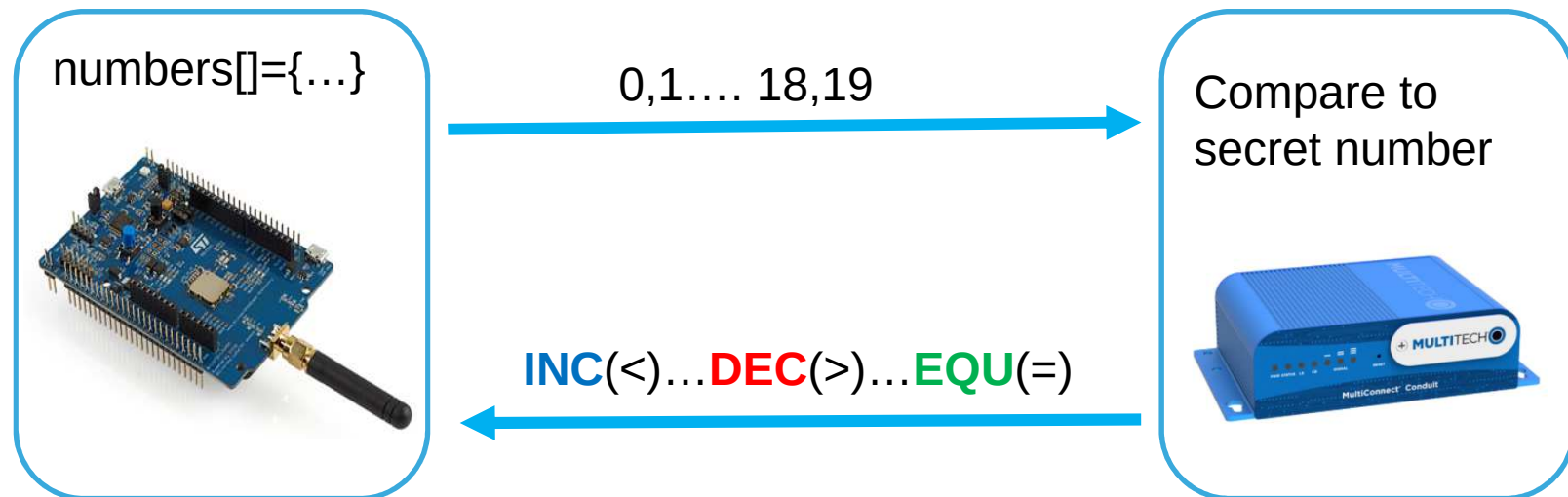
Hands-on

- Key learning

- Sending and receiving data from application layer using user defined actions both in RX and TX data call backs.
- Handling of application data buffer.
- User definition of software timer to indicate any event using LED.
- Debug of application

Hands-on

Number guess application



Number guess application

1. User enters an array of constant integer numbers within range: 0..20 in mote application source code, numbers order depends on user
2. Mote sends number by number to gateway every 10s
3. Gateway compares received number to secret number and send response to mote: (**I**)ncrease if $NB < SNB$, (**D**)ecrease if $NB > SNB$, (**E**)qual if $NB = SNB$
4. If mote receives (**E**)qual response green led lights, else blue or red led lights depends on (**I**)ncrease or (**D**)ecrease response has been received

Hands-on

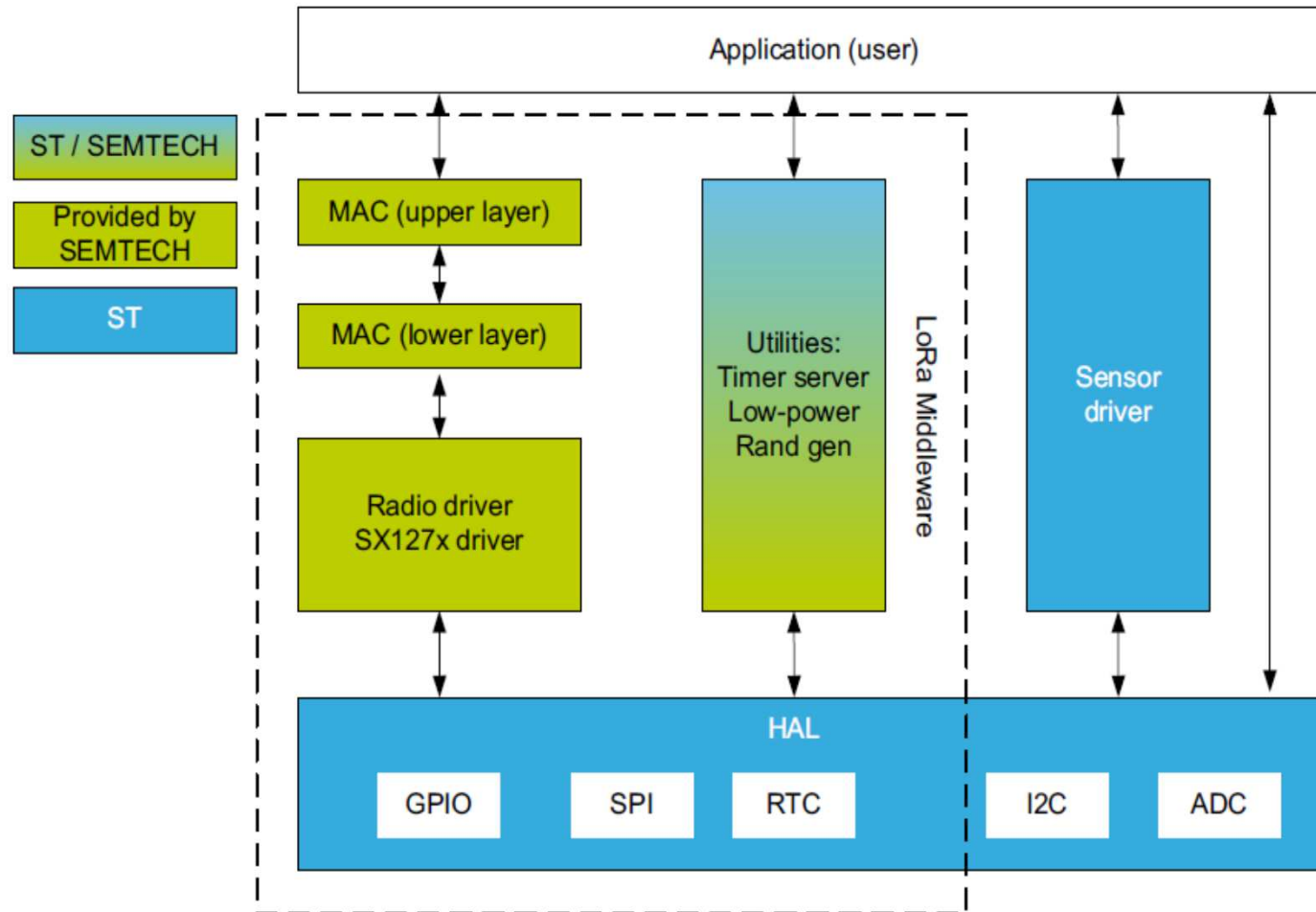
Architecture

Hints:

Detailed description in UM2073: “STM32 LoRa® software expansion for STM32Cube”

Hands-on

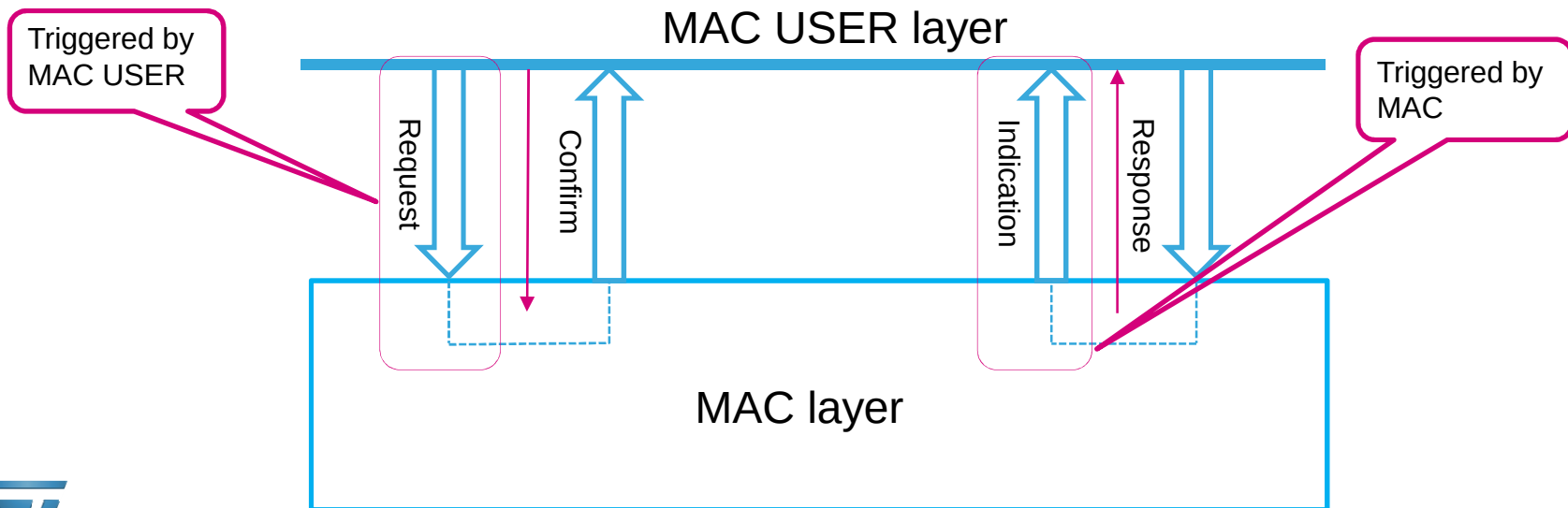
I-CUBE-LRWAN basics: architecture



IEEE 802.15.4 introduction: MAC API

I-CUBE-LRWAN protocol stack implementation base on IEEE 802.15.4 standard. Communications are passed between the MAC layer and the next highest stack layer (MAC user) by means of “service primitives”

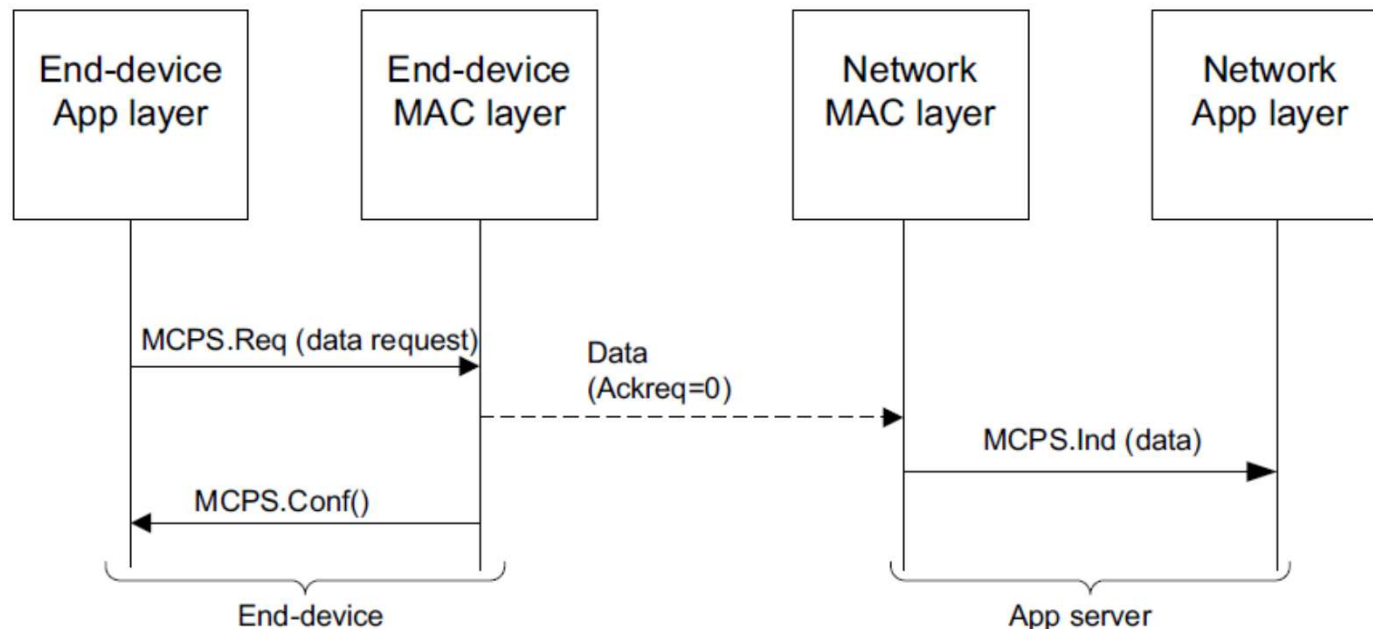
- Request ; initiated by the MAC user
- Confirm ; MAC user may solicit Confirm from the MAC layer
- Indication ; initiated by the MAC layer
- Response ; MAC layer may solicit Response from the MAC user



IEEE 802.15.4 introduction: MAC services

IEEE 802.15.4 MAC provides the interface to the application layer using two elements:

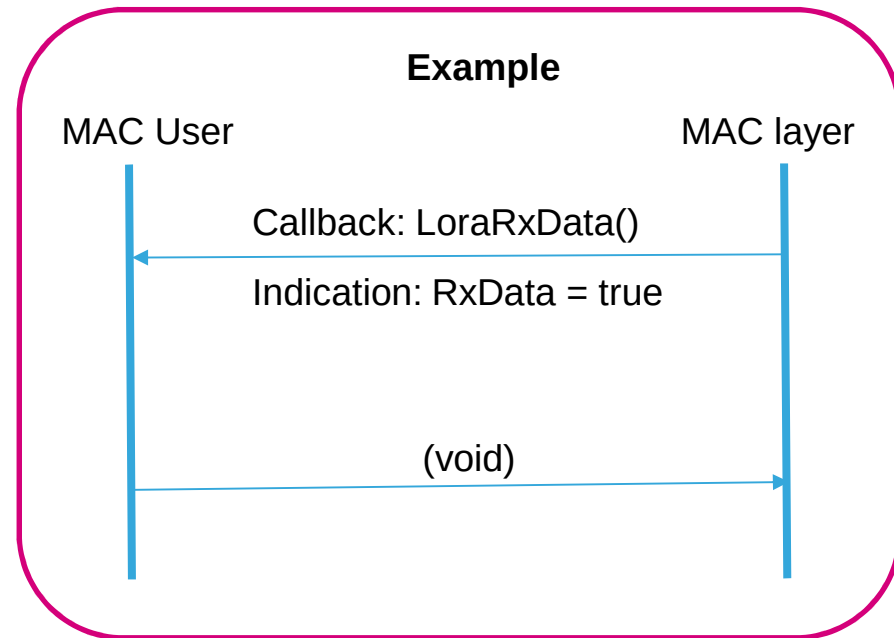
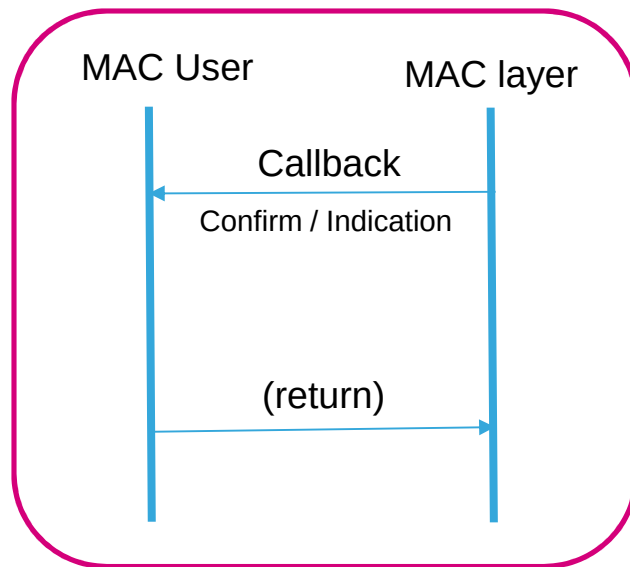
- MAC Management Service: this is called **MAC Layer Management Entity**, MLME. It provides the service through which layer management functions may be called or accessed. Example: JOIN network REQUEST,
- MAC Data Service: this is called the **MAC Common Port Service**, MCPS. It provides data transport services. Example: SEND data REQUEST.



Hands-on

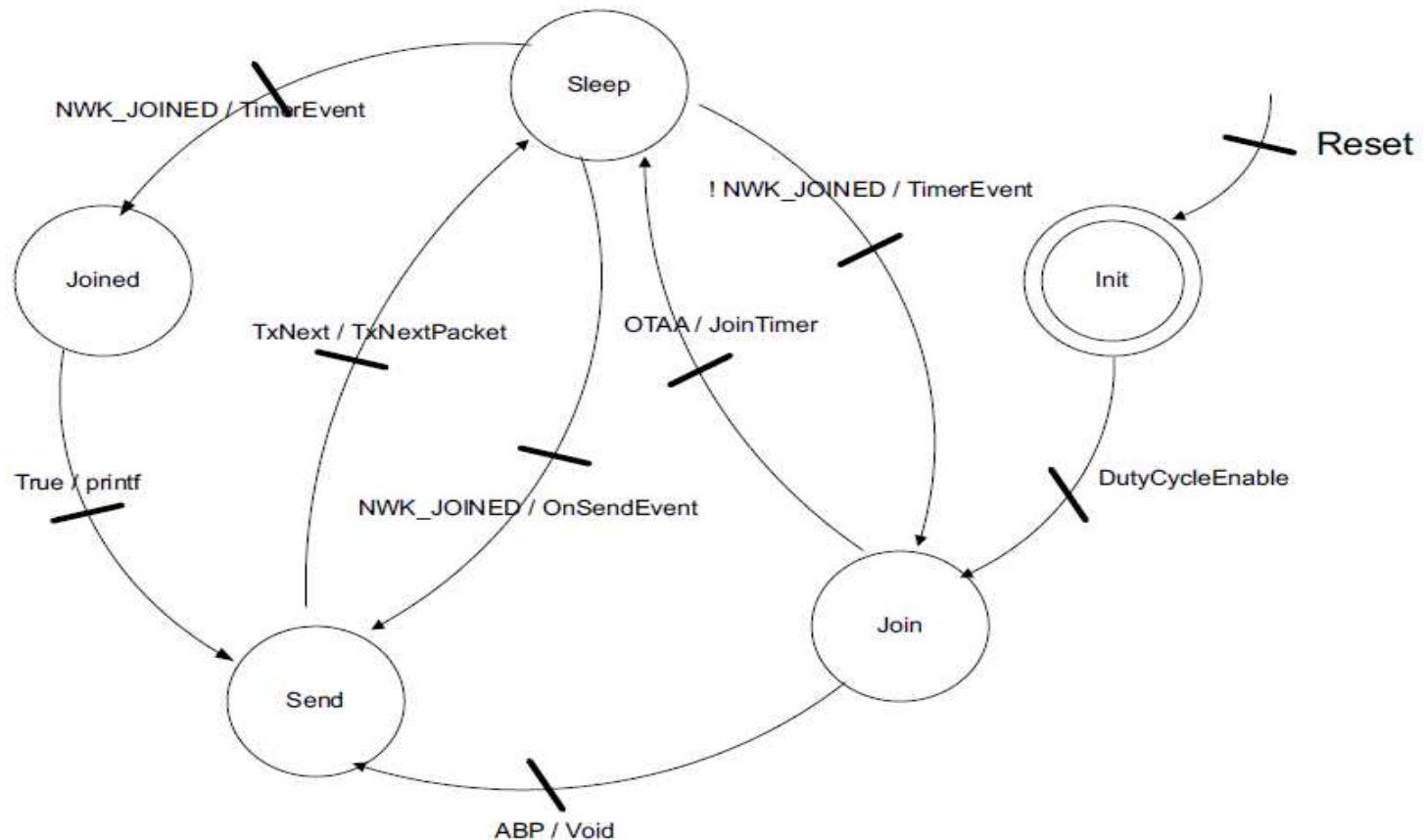
IEEE 802.15.4 introduction: callback mechanism

Request is issued by the MAC User by means of call to one of the MAC layer API functions. The most straightforward way for the MAC layer to reply (with Confirm and/or Indication) is via callback function.



I-CUBE-LRWAN basics: operation model

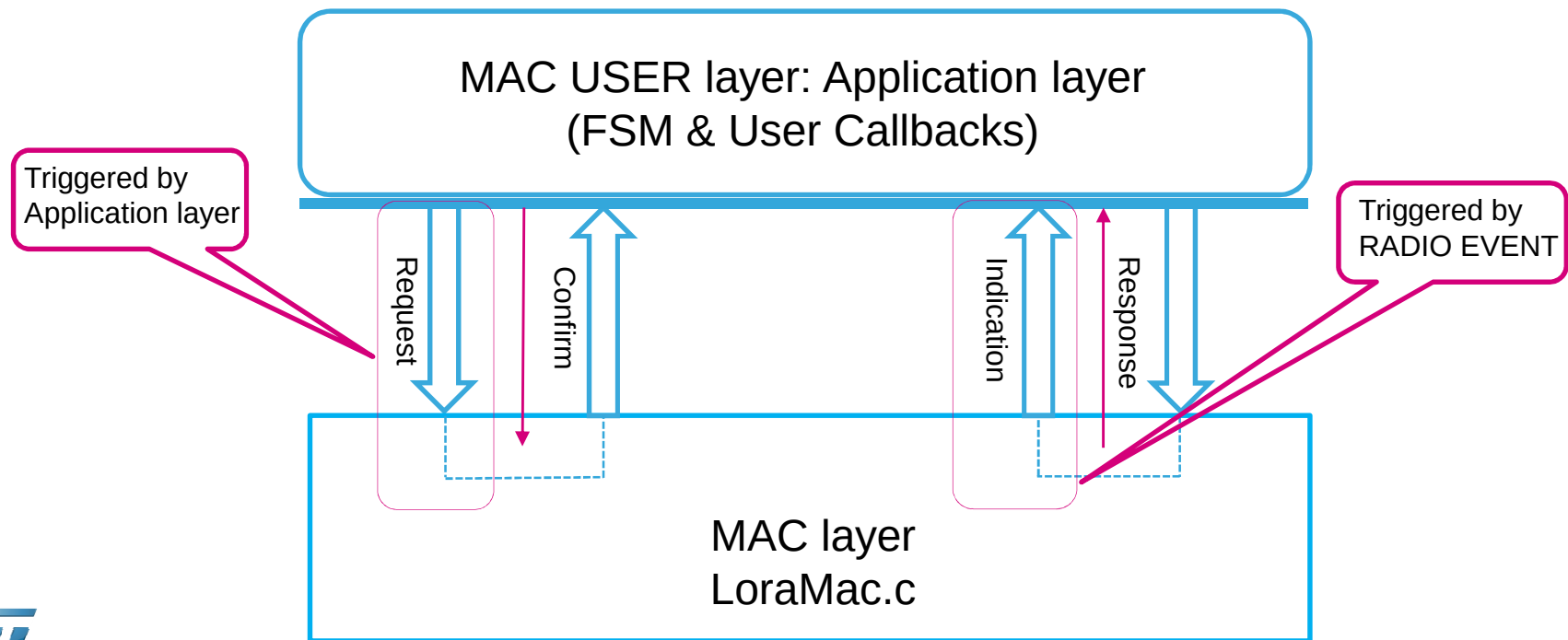
The model of operation proposed for this LoRa class A is based on **Finite State Machine (FSM)** running in the main thread of the application. LoRa FSM transition is triggered either by a timer event (Request) or by a radio event (Indication).



I-CUBE-LRWAN basics: operation model

IEEE 802.15.4 and operation model in practice:

- FSM requests services of MAC layer: Request / Confirm mechanism (i.e. send data)
- MAC layer indicates to FSM radio events and invokes related user callbacks (i.e. receive data event)



I-CUBE-LRWAN basics: fsm implementation

LoraWAN FSM is implemented in lora.c / lora.h source files

1. Initialization of the Lora class A FSM

```
void lora_Init (LoRaMainCallback_t *callbacks,  
LoRaParam_t* LoRaParamInit)
```

2. Lora class A FSM entry point

```
void lora_fsm (void)
```

3. Make Tx frame callback function, user is free to implement own code here

```
void LoraTxData (lora_AppData_t *AppData,  
FunctionalState* IsTxConfirmed)
```

4. Make Rx frame callback function, user is free to implement own code here

```
void LoraRxData (lora_AppData_t *AppData)
```

I-CUBE-LRWAN basics: fsm implementation

LoraWAN FSM is implemented in lora.c / lora.h source files

1. Get of the Lora class A FSM current state

```
DeviceState_t lora_getDeviceState( void );
```

Lora FSM states definition

```
typedef enum eDevicState  
{  
    DEVICE_STATE_INIT,  
    DEVICE_STATE_JOIN,  
    DEVICE_STATE_JOINED,  
    DEVICE_STATE_SEND,  
    DEVICE_STATE_CYCLE,  
    DEVICE_STATE_SLEEP  
} DeviceState_t;
```

I-CUBE-LRWAN basics: software timers

All the time dependences of I-CUBE-LRWAN are realized using software timer server `timeServer.c / timeServer.h`

1. Initialize the timer and associate callback

```
void TimerInit (TimerEvent_t *obj, void (*callback) (void))
```

2. Set the timer a timeout value on milliseconds

```
void TimerSetValue (TimerEvent_t *obj, uint32_t value)
```

3. Start the timer

```
void TimerStart (TimerEvent_t *obj)
```

4. Stop the timer

```
void TimerStop (TimerEvent_t *obj)
```

Hints:

User defined software timer can be used i.e. to control Tx and/or Rx led

I-CUBE-LRWAN basics: PRINTF redirection

PRINTF stream redirection to ST-Link Virtual Com Port was implemented in files vcom.c / vcom.h

1. Initialization of UART, default parameters: 115200,8,N,1
`void vcom_Init(void)`

```
UartHandle.Init.BaudRate      = 115200;  
UartHandle.Init.WordLength    = UART_WORDLENGTH_8B;  
UartHandle.Init.StopBits      = UART_STOPBITS_1;  
UartHandle.Init.Parity         = UART_PARITY_NONE;  
UartHandle.Init.HwFlowCtl      = UART_HWCONTROL_NONE;  
UartHandle.Init.Mode           = UART_MODE_TX_RX;
```

2. Redefined PRINTF() function can be used instead of printf() to output debug stream to Com Port terminal i.e.

```
PRINTF("\nNumber sent: %c\n\n", AppData->Buff[0]);
```

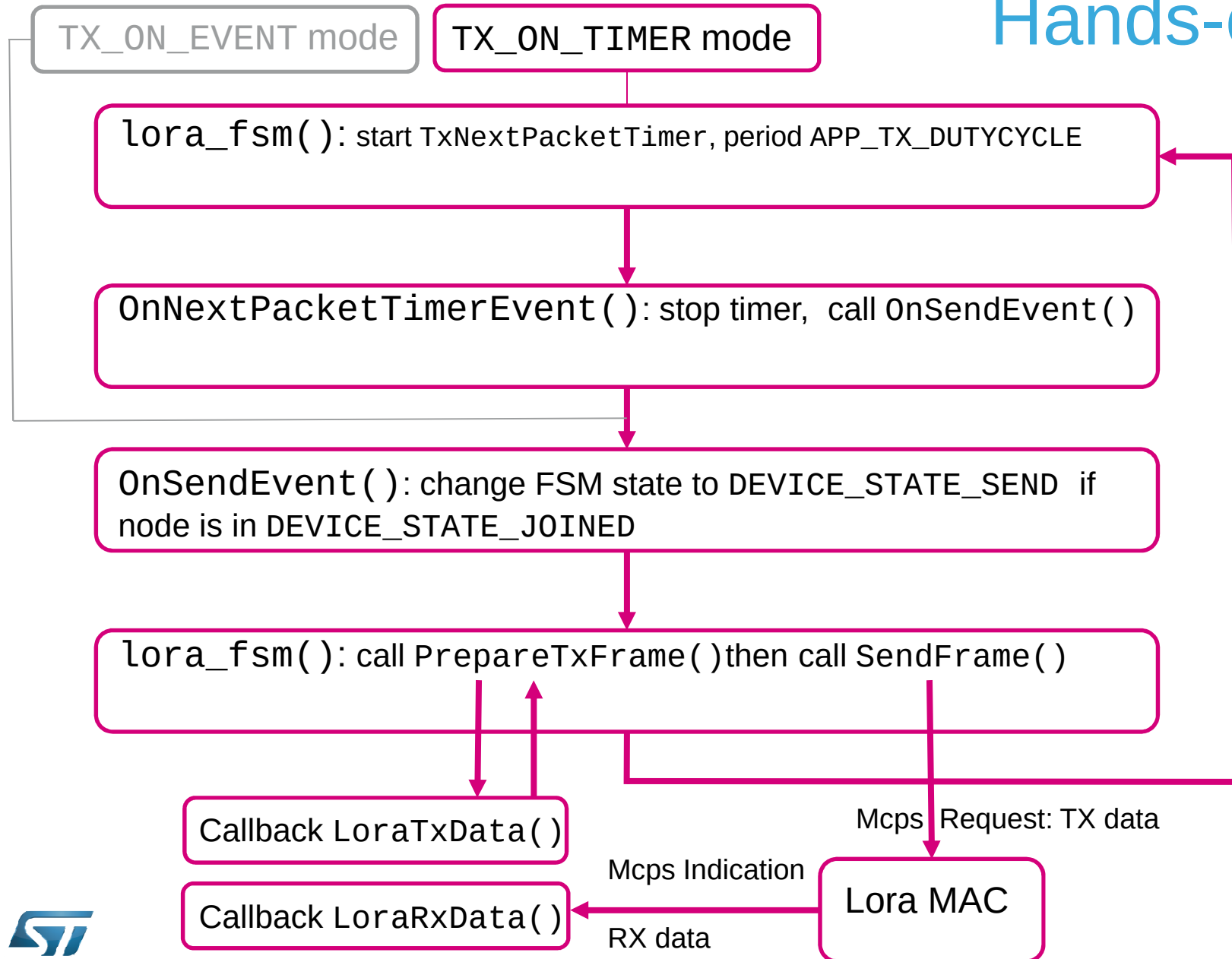
Hands-on

Implementation
How does it works ?

Hands-on

1. Lora mote sends data periodically to gateway according to TX_ON_TIMER initialization parameter, period is equal to APP_TX_DUTYCYCLE; main.c
2. All the process is handled by software timer TxNextPacketTimer and OnTxNextPacketTimerEvent() callback; lora.c
3. Uplink data send procedure is triggered by function OnSendEvent() called in timer callback; lora.c
4. FSM handles data send process by call PrepareTxFrame() and SendFrame() function, user can prepare own payload in LoraTxData() callback which is called by PrepareTxFrame() function; lora.c
5. Lora MAC takes control and call LoraRxData() callback, timing is handled by software timers as well; LoRaMac.c
6. It is possible to use TX_ON_EVENT mode and send uplink data asynchronously but OnSendEvent() function should be called in user defined callback function i.e. external interrupt

Hands-on



Hands-on

Lets start coding...

Hints:

Open related project

...\LORA_WS\Hands_on_v02\Projects\Multi\Applications\LoRa\End_Node_
number_guessing\MDK-ARM\B-L072Z-LRWAN1

Hands-on

Modification of **main.c**

`/* Private define --- */` section

1. Add definition of application data buffer size
`#define APP_DATA_BUF_SIZE 3`
2. Add definition of numbers buffer size
`#define NB_BUFSIZE 5`

Hands-on

Modification of **main.c**

```
/* Private variables --- */ section  
/* Private functions --- */ section
```

1. Add array of numbers to guess, numbers order depends on you

Please, rewrite below line and put here your numbers

```
const uint8_t numbers[NB_BUFSIZE]={5,1,3,7,2};
```

2. Add numbers array index variable

```
uint16_t nbidx = 0;
```

3. Add software timer to control Tx LED

```
static TimerEvent_t TxLedTimer;
```

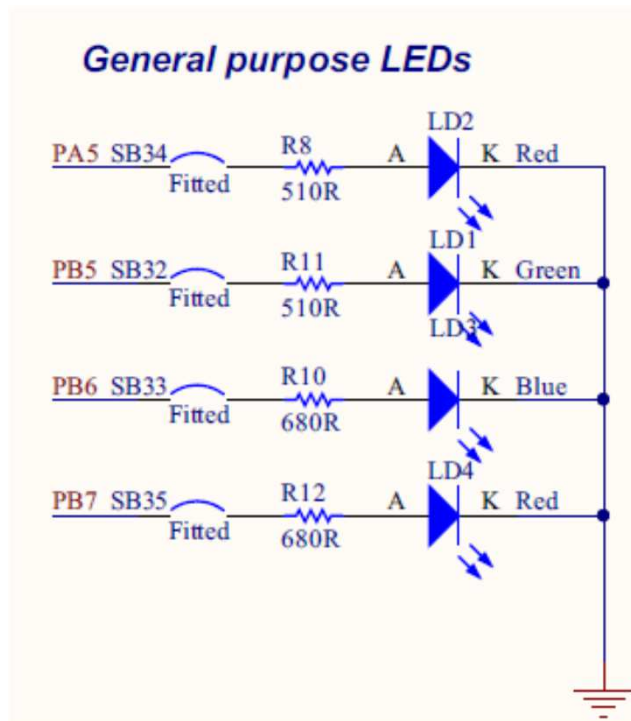
4. Add TxLedTimer callback to control Tx LED

```
static void OnTxLedTimerEvent( void );
```

Hands-on

Modification of **main.c**

Pin assignment of general purpose LEDs of B-L072Z-LRWAN1 Discovery board



Tx LED definition

LED_RED1 - LD2 - PA5

Application LEDs definition

LED_GREEN - LD1 - PB5

LED_BLUE - LD3 - PB6

LED_RED2 - LD4 - PB7

Modification of **main.c**

Hands-on

USER CODE BEGIN / END section

During workshop we can face quite high RF traffic and due to collisions avoidance, random RF transmit period should be introduced

```
srand1(HW_GetRandomSeed());  
uint32_t rndTxPeriod = randr(APP_TX_DUTYCYCLE, APP_TX_DUTYCYCLE * 2);  
LoRaParamInit.TxDutyCycleTime = rndTxPeriod;  
PRINTF("\nTX period: %dms\n\n", rndTxPeriod);
```

Hands-on

Modification of **main.c** **LoraTxData()** callback

```
/* USER CODE BEGIN 3 */

TimerInit(&TxLedTimer,OnTxLedTimerEvent);
TimerSetValue(&TxLedTimer,200); // Tx led on for 200ms
LED_On(LED_RED1);
TimerStart(&TxLedTimer);

AppData->Port = LORAWAN_APP_PORT;
*IsTxConfirmed = LORAWAN_CONFIRMED_MSG;

AppData->BuffSize = snprintf((char *)AppData->Buff,\
APP_DATA_BUF_SIZE,"%d",numbers[nbidx]);

PRINTF("\nNumber sent: %d\n\n",numbers[nbidx]);

/* USER CODE END 3 */
```


Hands-on

Modification of **main.c** LoraRxData() callback

```
/* USER CODE BEGIN 4 */
if (AppData->Port==LORAWAN_APP_PORT)
{
    switch (AppData->Buff[0])
    {
        case 'E':
            LED_On(LED_GREEN); LED_Off(LED_BLUE); LED_Off(LED_RED2);
            PRINTF("Number EQUAL.\n\n");
            break;
        case 'I':
            LED_On(LED_BLUE); LED_Off(LED_GREEN); LED_Off(LED_RED2);
            if (nbidx<NB_BUFSIZE-1) { nbidx++; }
            PRINTF("Number too LOW.\n\n");
            break;
        case 'D':
            LED_On(LED_RED2); LED_Off(LED_GREEN); LED_Off(LED_BLUE);
            if (nbidx<NB_BUFSIZE-1) { nbidx++; }
            PRINTF("Number too HIGH.\n\n");
            break;
        default:
            LED_Off(LED_GREEN); LED_Off(LED_BLUE); LED_Off(LED_RED2);
            PRINTF("Received data unknown.\n\n");
            break;
    }
    PRINTF("\n");
}
/* USER CODE END 4 */
```

Modification of **main.c** `OnTimerLedEvent()` callback

1. Add `OnTimerLedEvent()` callback to switch off Tx led when related software timer has expired

```
static void OnTxLedTimerEvent( void )  
{  
    LED_Off(LED_RED1);  
}
```

Hands-on

Modification of **lora.c** Duty cycle control feature

1. Switch off duty cycle control

```
#define LORAWAN_DUTYCYCLE_ON          false
```

Hints:

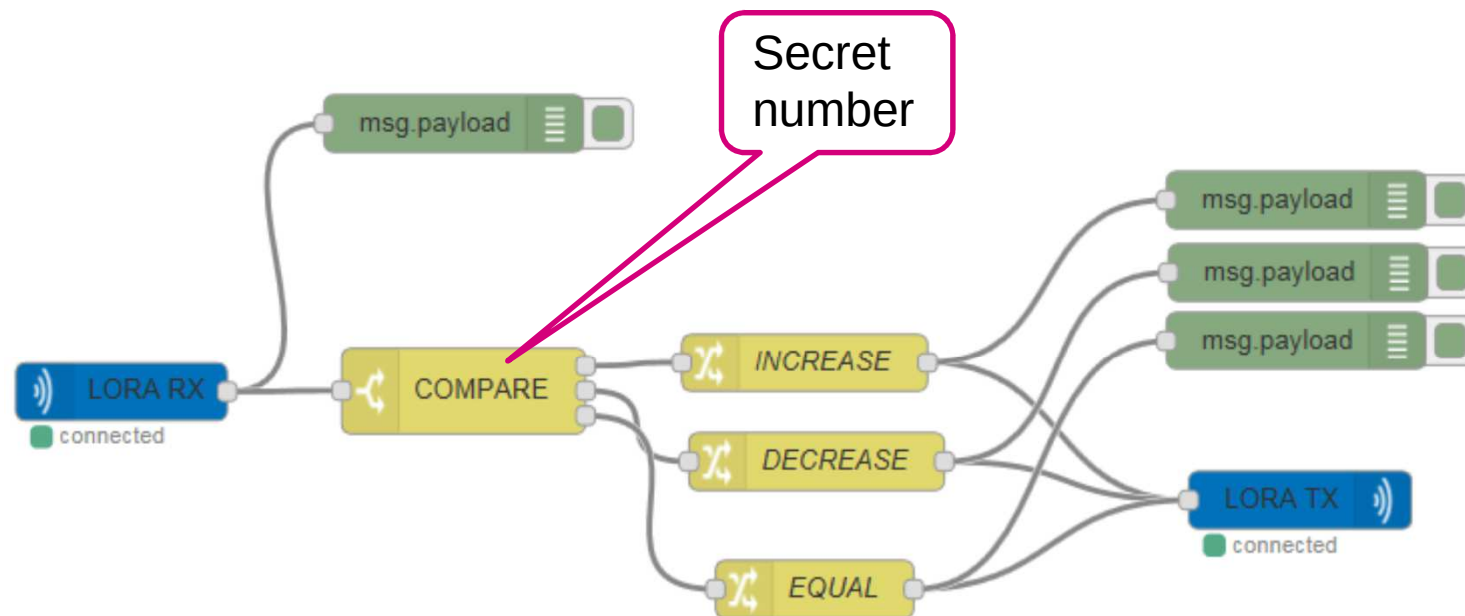
According to EN300220-1 there is a duty cycle limitation in ISM band: *“In a period of 1 hour the duty cycle shall not exceed the spectrum access and mitigation requirement values...”*. For 868MHz sub-band used by Lora it is 1%. LoraWAN protocol stack controls duty cycle but for our hands-on we can switch it off due to practical reasons (quite high on-air traffic).

Hands-on

Lets compile and download binary...

Hands-on

Application server functionality



Hands-on

Terminal output (115200,8,N,1): joining

```
OTAA
DevEui= 34-31-37-32-7A-36-79-0D
AppEui= 01-01-01-01-01-01-01-01
AppKey= 2B 7E 15 16 28 AE D2 A6 AB F7 15 88 09 CF 4F 3C

VERSION: 44011000

INIT
JOINING...
txDone
rxTimeOut
rxTimeOut
txDone
rxDone
JOINED ← Lora mote already joined
Number sent: 0
txDone ← Green Tx led blinks
rxDone
Number too LOW.
```

Hands-on

Terminal output (115200,8,N,1): sending numbers

```
Number sent: 4
txDone
rxDone
Number too LOW. ← Blue led on

[00]
Number sent: 9
txDone
rxDone
Number too HIGH. ← Red led on

[00]
Number sent: 5
txDone
rxDone
Number EQUAL. ← Green led on
```

Hands-on

Thank you